

Artificial Intelligence and Data Science

CSA1008 – Software Engineering

Assignment Title: CO4-AT2-CI/CD Pipeline Design Exercise

Case Study Title: Smart Waste Collection System

(SDG 11: Sustainable Cities and Communities)

Name: K. Dedeepya

Reg No: 192324252

Class & Section: 4th year B. Tech AIDS

Faculty Name: Sharmili K

Date of Submission: 20-08-26

CI/CD Pipeline Design for Smart Waste Collection System:

1. Introduction

Municipal waste collection systems commonly use fixed collection schedules without considering the actual amount of waste present in each bin. This can result in overflowing bins, unnecessary vehicle trips, increased fuel consumption, and inefficient utilization of collection resources. The proposed Smart Waste Collection System is a software solution that monitors bin status, identifies bins requiring collection, supports route optimization, and provides a dashboard for municipal authorities and collection personnel.

Since the system handles continuously changing operational information, software updates must be delivered safely and reliably. A Continuous Integration and Continuous Deployment (CI/CD) pipeline is therefore proposed to automate code integration, building, testing, quality checking, security validation, packaging, staging, deployment, and monitoring.

2. Problem Analysis

The existing fixed-schedule approach creates several challenges:

- Waste bins may overflow before the scheduled collection.
- Nearly empty bins may be collected unnecessarily.
- Collection vehicles may travel inefficient routes.
- Fuel and operational costs may increase.
- Manual monitoring can delay identification of critical bins.
- Software failures can affect collection operations.

The proposed system addresses these issues by providing:

- Bin-status monitoring
- Collection-priority identification
- Route optimization
- Vehicle and collection tracking
- Administrative dashboard
- Alerts for bins requiring immediate attention
- Collection history and reports

The CI/CD pipeline ensures that improvements to these functions can be tested and delivered without compromising system reliability.

3. CI/CD Requirements

The Smart Waste Collection System requires a CI/CD pipeline with the following capabilities:

Requirement	CI/CD Support
-------------	---------------

Frequent source-code changes	Git-based version control
Reliable application build	Automated build
Correct waste-processing logic	Unit testing
Reliable communication between modules	Integration testing
Correct API behaviour	API testing
Reliable dashboard functionality	UI testing
Good source-code quality	SonarQube analysis
Protection against vulnerable dependencies	Security scanning
Consistent application packaging	Docker
Safe pre-production validation	Staging environment
Controlled production release	Deployment approval
Quick recovery from failures	Rollback
Continuous operational observation	Monitoring and logging

4. Proposed Software Solution

The proposed system consists of interconnected software components.

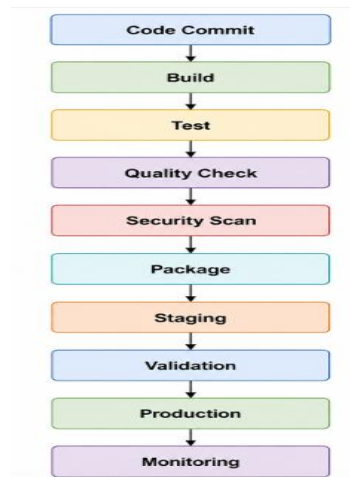
Main System Components

- 1. Bin Monitoring Module:** Receives and processes the current fill-level/status information of waste bins.
- 2. Waste Management Backend:** Processes bin information, maintains collection records, and provides APIs for other system components.
- 3. Route Optimization Module:** Uses bin locations, collection priority, vehicle availability, and other relevant information to determine an efficient collection route.
- 4. Municipal Dashboard:** Allows authorities to view bin status, collection requirements, vehicle activities, routes, and reports.
- 5. Collection Personnel Interface:** Allows collection personnel to view assigned routes and update collection status.
- 6. Notification Module:** Generates alerts when bins require collection or when abnormal conditions are detected.

These components are mentioned only to define what the CI/CD pipeline will build, test, and deploy.

5. Proposed CI/CD Pipeline

The proposed pipeline follows:



The pipeline automatically validates every important software change before it reaches the live system.

6. CI/CD Pipeline Architecture

Main Pipeline Diagram



This will make the architecture easy to understand during evaluation.

7. Pipeline Stage 1 – Source Code Management

Git and GitHub are used to manage the source code of the Smart Waste Collection System.

The repository can contain components such as:

- Backend services
- Route optimization logic
- Dashboard
- Collection interface
- Testing files
- Configuration files

A developer works on a feature branch and submits the changes for review before merging them into the main branch.

Purpose

- Maintains version history.
- Supports collaboration.
- Enables code review.
- Prevents uncontrolled changes.
- Provides previous versions for recovery.

8. Pipeline Stage 2 – Continuous Integration

Whenever new code is committed or a Pull Request is created, the CI pipeline is triggered automatically.

The pipeline performs:

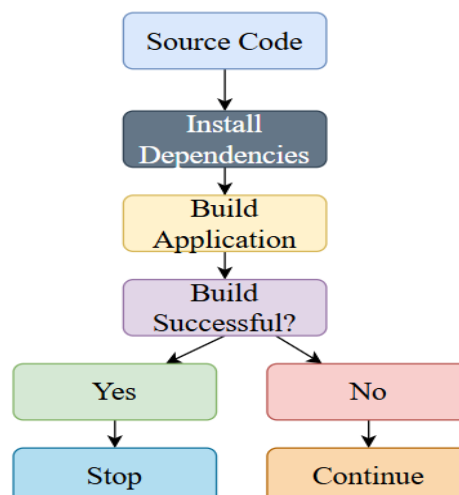
1. Source-code checkout
2. Dependency installation
3. Application build
4. Automated testing
5. Code-quality analysis
6. Security checks

If a critical check fails, the pipeline stops before the software can progress to deployment.

9. Pipeline Stage 3 – Build

The application is automatically built after successful source-code validation. The build process verifies that the different software components can be assembled successfully.

Build Flow



This prevents incomplete or broken builds from entering later pipeline stages.

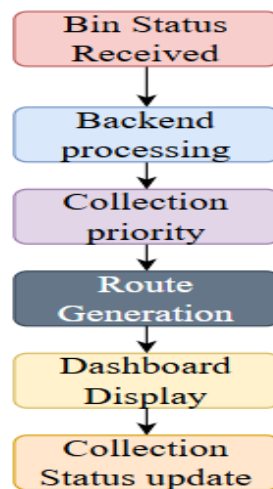
10. Pipeline Stage 4 – Automated Testing

Automated testing is important because incorrect software could generate incorrect collection priorities or routes.

Testing Strategy

Testing Type	Application Area	Purpose
Unit Testing	Bin processing	Verify individual functions
Unit Testing	Route logic	Verify route calculations
Integration Testing	Backend + Database	Verify data interaction
API Testing	Backend APIs	Verify requests and responses
UI Testing	Admin dashboard	Verify important user functions
End-to-End Testing	Complete workflow	Validate the complete system
Regression Testing	Existing features	Ensure new changes do not break old features

Example Application Workflow Tested



The CI/CD pipeline verifies this workflow automatically before deployment.

11. Pipeline Stage 5 – Code Quality Analysis

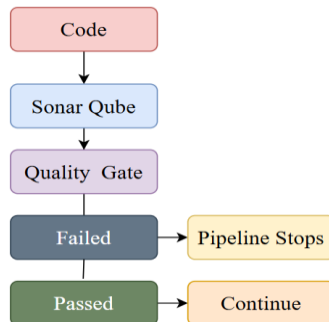
SonarQube can be integrated into the pipeline to evaluate source-code quality.

It can identify:

- Bugs
- Code smells
- Duplicate code
- Maintainability issues

- Security-related coding issues

A **Quality Gate** is used to decide whether the application can continue.



This prevents poor-quality code from progressing toward production.

12. Pipeline Stage 6 – Security Validation

Security checks are performed before deployment.

Security Checks

Check	Purpose
Secret Detection	Prevent passwords and API keys from being committed
Dependency Scanning	Detect vulnerable libraries
Static Analysis	Identify insecure coding patterns
Container Scanning	Detect vulnerabilities in Docker images
Authentication Testing	Verify protected system functions
Authorization Testing	Prevent unauthorized access

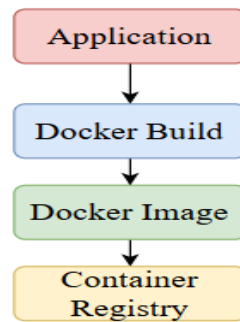
Suggested Tools

- SonarQube
- OWASP Dependency-Check
- Trivy
- GitHub security features

If a critical vulnerability is identified, the pipeline should block the release.

13. Pipeline Stage 7 – Application Packaging

After successful testing and security validation, the application is packaged using Docker.



A version number is assigned to each image.

For example:

- smart-waste: v1.0
- smart-waste: v1.1
- smart-waste: v1.2

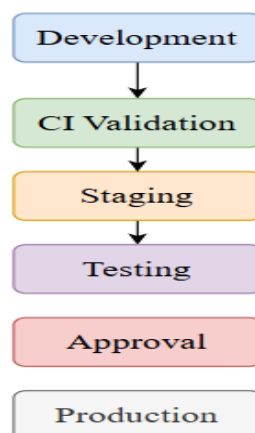
This makes application versions easy to identify and supports rollback.

14. Deployment Environments

The pipeline uses three major environments.

Environment	Purpose	Pipeline Activity
Development	Feature development	Initial validation
Staging	Production-like testing	Integration and E2E testing
Production	Live municipal system	Controlled deployment

Environment Flow



The production environment should receive only versions that successfully pass the required quality and security checks.

15. Deployment Strategy

A Rolling Deployment strategy is suitable for this system.

Instead of replacing every running application instance simultaneously, the new version is introduced gradually.

Advantages

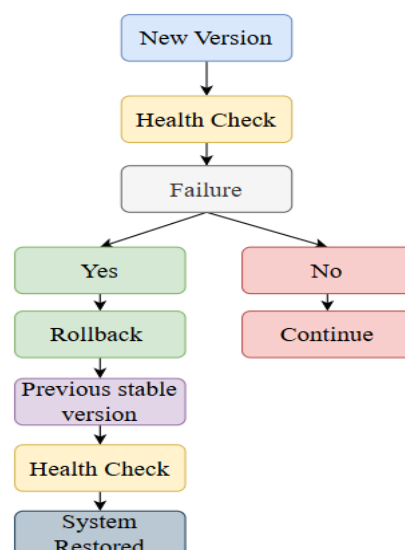
- Reduces downtime.
- Allows gradual deployment.
- Limits the impact of failures.
- Supports quick recovery.

16. Rollback Strategy

Because the system supports an important municipal service, the previous stable version should always be retained.

If the newly deployed version causes problems such as API failures, incorrect processing, or application unavailability, the system can return to the previous stable version.

Rollback Flow



Example

Current Version → v1.2

Previous Version → v1.1

If v1.2 fails production validation, the system can restore v1.1.

17. Monitoring and Notifications

After deployment, the system should be continuously monitored.

Monitoring Areas

- Application availability
- API response time

- Server/container health
- Database availability
- Failed requests
- Route-processing failures
- Collection service status
- CPU and memory usage

Prometheus can collect system metrics and Grafana can present them through dashboards.

Notifications can be generated when:

- Build fails
- Tests fail
- Quality gate fails
- Security scan detects critical problems
- Deployment fails
- Production health checks fail

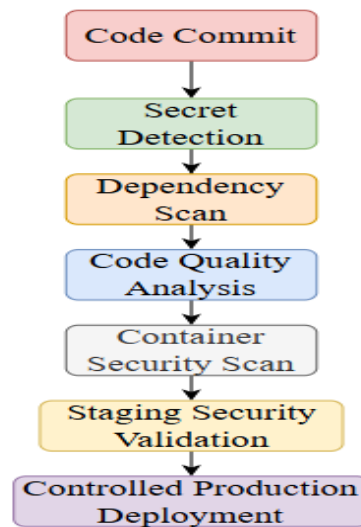
18. Tools and Technology Selection

Pipeline Activity	Tool	Justification
Version Control	Git	Tracks changes and versions
Source Repository	GitHub	Supports collaboration and code review
CI/CD	GitHub Actions	Automates pipeline stages
Build	npm / Maven / appropriate build tool	Automates application builds
Unit Testing	PyTest / JUnit	Automates functional tests
API Testing	Postman / Newman	Validates backend APIs
UI/E2E Testing	Selenium	Validates user workflows
Code Quality	SonarQube	Detects quality issues
Dependency Security	OWASP Dependency-Check	Identifies vulnerable dependencies
Container Security	Trivy	Scans container images
Packaging	Docker	Provides consistent deployment
Container Registry	GitHub Container Registry	Stores versioned images

Deployment	Kubernetes / Docker	Supports controlled deployment
Monitoring	Prometheus	Collects system metrics
Visualization	Grafana	Displays monitoring information
Notification	GitHub Actions / Email	Communicates pipeline results

19. Security Strategy

Security is integrated throughout the CI/CD lifecycle rather than being treated as a final activity.



Security Practices

- Use secure repository access.
- Store credentials in protected secrets.
- Never hard-code passwords or API keys.
- Use HTTPS for application communication.
- Apply role-based access control.
- Follow least-privilege principles.
- Scan dependencies regularly.

20. Pipeline Failure Handling

Pipeline Failure	Action
Build Failure	Stop pipeline and notify developer
Unit Test Failure	Reject the build
Integration Test Failure	Stop deployment
Quality Gate Failure	Block release

Critical Security Issue	Block deployment
Staging Test Failure	Prevent production deployment
Production Health Failure	Trigger rollback
Deployment Failure	Restore previous stable version

21. Benefits of the Proposed CI/CD Pipeline

The proposed pipeline provides:

1. Faster and safer software delivery.
2. Automated validation of code changes.
3. Early detection of defects.
4. Improved source-code quality.
5. Better security before deployment.
6. Consistent application packaging.
7. Reduced deployment errors.
8. Reduced production downtime.
9. Faster recovery using rollback.
10. Continuous monitoring of the application.

22. Contribution to SDG 11

The Smart Waste Collection System contributes to SDG 11 – Sustainable Cities and Communities by supporting efficient and reliable urban waste-management services.

The system can help authorities identify waste-collection requirements, reduce unnecessary collection trips, improve route planning, and make better use of municipal resources. The CI/CD pipeline further ensures that improvements to the system can be introduced in a controlled, secure, and reliable manner.

23. Conclusion

The proposed CI/CD pipeline provides a structured DevOps approach for the Smart Waste Collection System. It integrates source-code management, automated building, testing, code-quality analysis, security validation, containerized packaging, staging, controlled production deployment, monitoring, and rollback. Only the essential project components are considered so that the pipeline can define what needs to be built and tested without turning the assessment into a full software implementation document. By combining automation, testing, security, monitoring, and reliable deployment practices, the proposed pipeline improves the quality, security, reliability, and maintainability of the Smart Waste Collection System while supporting efficient municipal waste-management operations.